esprit Se former autrement HONORIS UNITED UNIVERSITIES	**EXAMEN** Semestre : 1 ■ 2 □ Session : Principale ■ Rattrapage □

Unité d'enseignement : Equipe Web
Module (s) : Architecture Orientée Services (SOA)
Classe(s): 4A

Nombre des exercices : 3 Nombre de pages : 5 pages+ 1feuille de réponse

Date : 12/01/2024 Heure : 16h Durée : 1h30

NB : Les étudiants sont demandés de répondre directement sur la feuille de réponse.

Exercice 1 : (6pts)

1. Définir brièvement les termes suivants: (2pts)
   - Paradigme
   - Middleware
   - Service web
   - Interopérabilité
2. Soit la liste des termes ci-dessous. (3pts)

TCP – Intégration – Deux – WSDL –JWT- UDDI – SOAP – RPC – Middleware – Trois – Requête – Réponse – status – XML- Cinq .

On vous demande de compléter les phrases suivantes par des termes de la liste.

- Afin de mettre en place un service web étendu [1]…….. éléments sont nécessaires.
- [2]……. est utilisé pour le référencement des services web.
- Le protocole [3]……. est utilisé pour la communication des services web étendus.
- Il faut utiliser le langage [4]…….. afin d'obtenir la description d'un service web.
- L'utilisation de [5] …….. est courante pour assurer l'authentification et l'autorisation dans les applications web sécurisées.
- [6]…….constitue un mécanisme permettant à un programme d'appeler des fonctions ou méthodes distantes.

3. Soit le document XML ci-dessous (1pt)

```
<evenement>
<type>Sport</type>
<date>13/01/2024</date>
<description>Coupe d'affrique</description>
</sport>
</evenement>
<evenement>
<type />
<Description>Visite culturelle</description>
<date>15/03/2024</date>
</evenement>
```

Chercher et corriger les erreurs afin d'obtenir un document XML bien formé.

Exercice 2: (8pts)

Partie 1 (2pts)

Soit la méthode inscriptionEtudiant (id, nomPrenom, niveau, option), On souhaite consommer cette méthode via une requête SOAP.

1. Compléter la requête/réponse SOAP permettant de consommer l'opération inscriptionEtudiant.

Requête

```
<soap-env:Envelope
xmlns:soapenv=http://schemas.xmlsoap.org/soap/envelope xmlns: esp="http://examen.esprit.tn/">
<soapenv:Header/>
<…….>
<…….……>
<id>125</id>
<nomPrenom>Anim makni</nomPrenom>
<niveau>125</niveau>
<option>4TWIN</option>
<……..…>
<……..>
</soapenv:Envelope>
```

Réponse

```
<soap-env:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope">
<……..>
<…….. xmlns:ns2="http://examen.esprit.tn/">
<NumInscription> 1J256AB99C </NumInscription >
<……>
</……..>
</soapenv:Envelope>
```

Partie2(6pts) : Système de Gestion Scolaire

Le système de gestion scolaire est une plate-forme éducative permettant de gérer les cours, les programmes, les enseignants qui vont assurer ces cours, et les étudiants qui vont y participer.

-Un cours est caractérisé par un identifiant unique et un titre. Chaque cours est assuré par un enseignant et peut avoir plusieurs étudiants inscrits, étant lié à un programme spécifique.

-Un enseignant est caractérisé par un identifiant, un nom et une adresse e-mail. Chaque enseignant peut assurer un ou plusieurs cours.

-Un étudiant est caractérisé par un identifiant, un nom, un niveau d'études et une option. Chaque étudiant peut s'inscrire à un ou plusieurs cours.

-Un programme est caractérisé par un identifiant unique, un nom et une date. Chaque programme englobe un ou plusieurs cours.

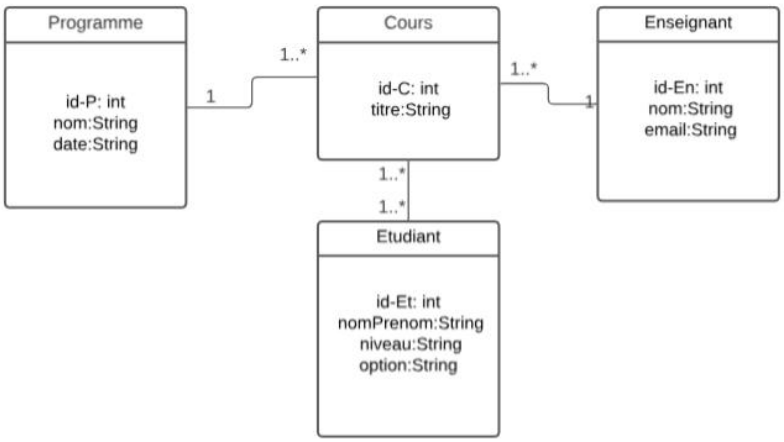


Figure 1: Diagramme de classes proposé

Veuillez répondre aux questions de la partie 2 en se basant sur le diagramme présenté dans la figure 1.

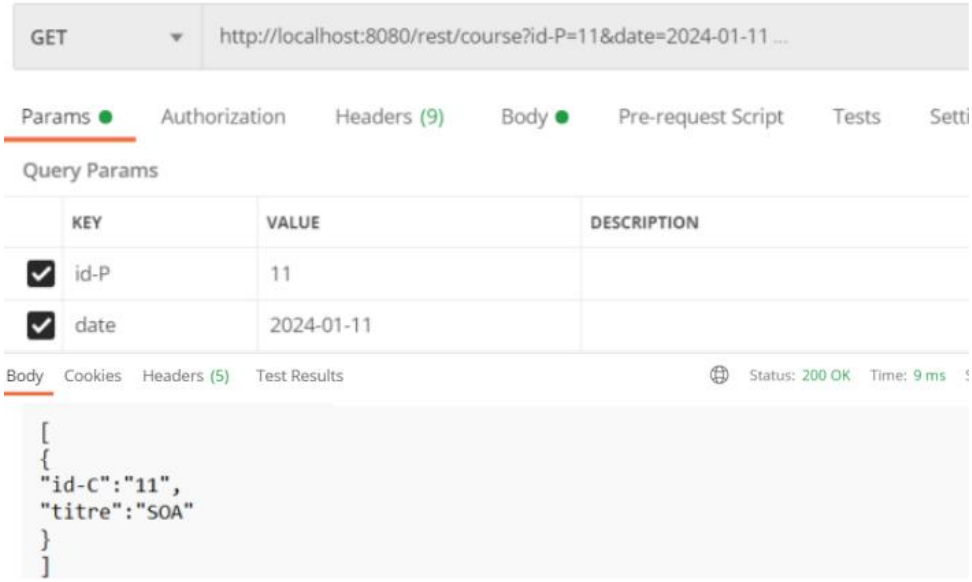
1. Soit le code présenté ci-dessous. On désire ajouter un cours : (1,5 pt)

```
@POST
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.TEXT_PLAIN)
public Response addCourse(Cours c)  {
        if (cours.add(c))   \\ cours est la liste des cours

                return Response.status(Status.CREATED).entity("cours ajouté").build();

        return Response.status(Status.NOT_FOUND).build();
}
```

Remplir le tableau sur la feuille de réponse.

Requête	POST	/rest/course
	Header	[1] ……..
	Body	{ "id-C": "11", "titre": "SOA" }
Réponse	Header	[2]……..
	Status	[3]……..
	Body	[4]……

2. La méthode testée au niveau du postman et présentée dans la figure ci-dessous permet de chercher un ou plusieurs cours programmé(s) selon l'id-P et la date (2.5pts)



En se basant sur le résultat du test sur postman, remplir le tableau ci-dessous

Requête	GET	/rest/course [5]……
Réponse	Header	[6]…….
	Status	[7]……
	Body	[8]……..

3. On souhaite récupérer la liste des étudiants qui vont participer à un cours. Lors du test postman, le résultat affiche une erreur (2pts)

```
@GET
@Produces(MediaType.TEXT_PLAIN)
@Path("{id}")
public Response displayStudentsListe(@PathParam ("id") String id-Et)  {
.....
.....
                    return Response.status(Status.CREATED).entity(students).build();
\\ students est la liste des étudiants

          return Response.status(Status.NOT_FOUND).build();
}
```

a. Donner le statut résultat de cette méthode et expliquer la source de cette erreur
b. Proposer une solution

Exercice3(6pts) :

Vous êtes en charge du développement d'une API GraphQL pour mettre en œuvre le système de gestion scolaire qui implémente le diagramme ci-dessous (fig1).

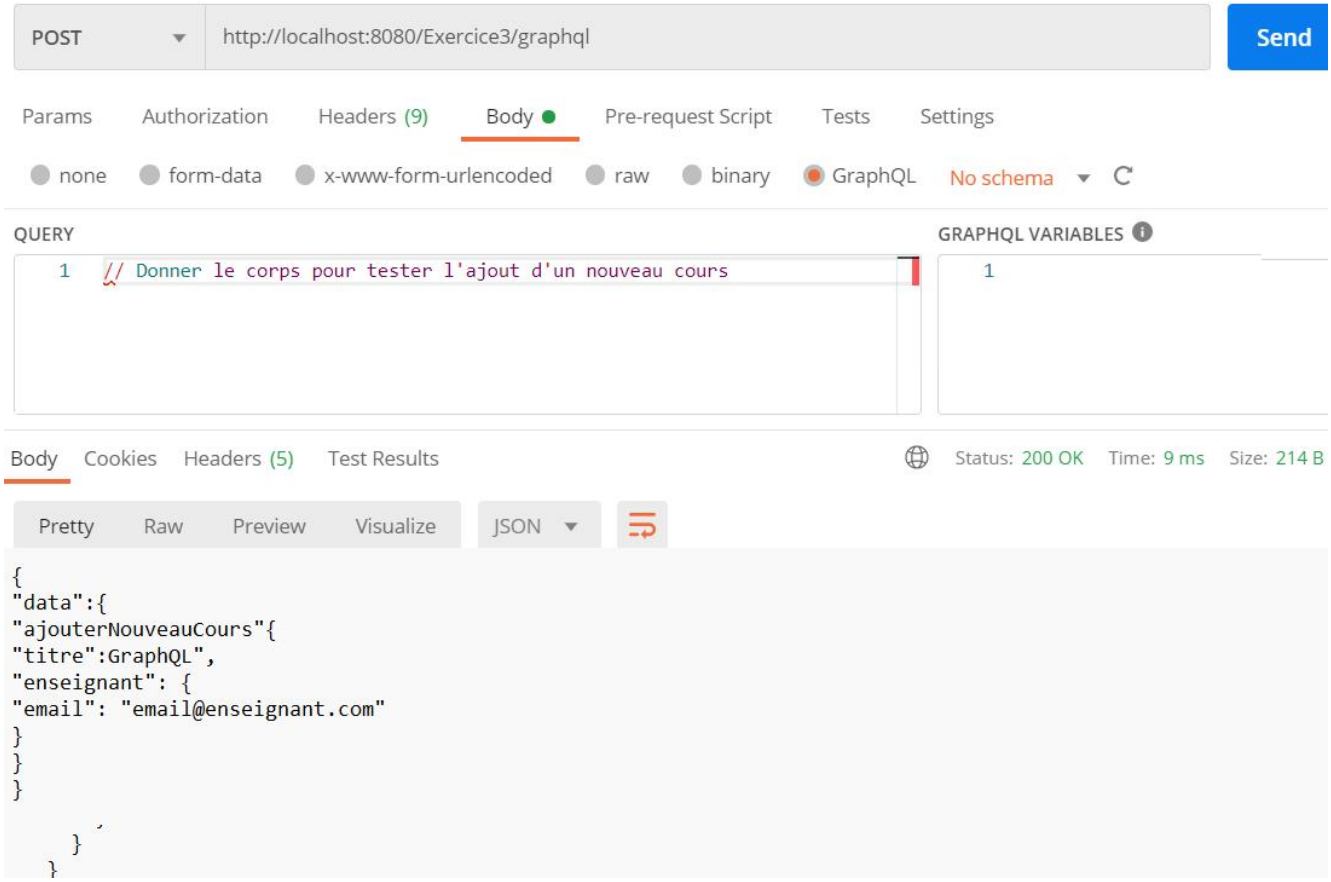
1. Compléter le schéma GraphQL pour représenter les types d'entités ainsi que les opérations suivantes : (3,5pts)

- "etudiantsInscritsAuCours " : une opération qui permet d'afficher la liste des étudiants inscrits à un cours spécifique.
- "detailsEnseignantDuCours" : une opération qui permet d'afficher les détails d'un enseignant qui enseigne un cours particulier.
- "ajouterNouveauCours" : une opération qui permet d'ajouter un nouveau cours avec un titre spécifique et l'id d'un enseignant existant et qui retourne la liste des cours, en affichant uniquement le titre du cours et l'email de l'enseignant associé.

```
type Programme {
 id-P: Int!
 nom: String!
 date : String!
 cours:[1] ............  }

type Cours {
 id-C: Int!
 titre: String!
 enseignant:[2] .......................
 etudiants:[3]  .......................  }

type Enseignant {
 id-En: Int!
 nom: String!
 email: String!
 coursEnseignes:[4]. ......................  }

type Etudiant {
 id-Et: Int!
 nomPrenom: String!
 niveau: String!
 option: String!
 coursInscrits: [5]...................................... }
```

```
type [6]..................................................................
{
[7]..................................................................
[8]..................................................................
}

type[9] ..................................................................{
[10]..................................................................
}
[11]................ {
query:Qscolaire
mutation:Mscolaire
}
```

2. Écrire le corps de la requête GraphQL nécessaire pour créer un nouveau cours, générant ainsi les résultats illustrés dans l'application Postman, tels qu'indiqués dans la figure ci-dessous (1pt)



3. Expliquer en quoi consiste le principe de mutation dans GraphQL.(0,5pt)
4. Donner le rôle des résolveurs dans API GraphQL, puis expliquer le rôle de l'implémentation du GraphQL Root Resolver **<u>brièvement</u>** .(1pts)

Bon Travail ☺